

Connexion de la borne IoT Arduino au projet Moodbar

Rapport détaillé de la session du 4 août 2026 — Projet Moodbar, Dakar Institute of Technology

1. Contexte et objectif de la soirée

En amont de cette session, l'équipe avait conçu et câblé physiquement un prototype matériel pour la borne IoT du projet Moodbar (« MoodMap DIT ») : un montage Arduino Uno avec écran LCD, trois boutons (Heureux / Neutre / Triste), trois LEDs de couleur et un buzzer. Jusque-là, la borne IoT du projet Moodbar était **simulée en logiciel** uniquement (une page web en mode kiosque), conformément à une décision documentée dans `REQUIREMENTS.md` (contrainte budget, pas de matériel physique).

L'objectif de cette session était de **faire remplacer cette simulation logicielle par le vrai prototype matériel**, c'est-à-dire :

1. Acter le changement de décision dans la documentation du projet.
2. Écrire le firmware Arduino et le pont logiciel (« bridge ») nécessaires pour relier la borne physique au backend existant.
3. Câbler, flasher et tester le tout **en conditions réelles**, jusqu'à voir un vote physique atterrir dans la base de données.

2. Décisions prises en amont de l'implémentation

Question	Décision retenue	Raison
Comment la borne envoie-t-elle les votes au backend ?	Via l'API HTTP existante (<code>POST /api/votes</code>)	Cohérent avec l'architecture actuelle (« deux points de collecte alimentent le même backend »), pas de duplication de logique base de données, pas d'identifiants MySQL nécessaires sur la machine reliée à l'Arduino

Que devient <code>kiosk.html</code> (la borne simulée) ?	Conservée comme solution de secours logicielle	Utile pour une démo sans matériel, ou en développement avant que le boîtier physique soit finalisé
--	--	--

3. Fichiers créés et documentation mise à jour

3.1 Nouveau dossier `moodbar-iot/`

Fichier	Rôle
<code>firmware/moodbar_kiosk.ino</code>	Sketch Arduino : gestion des boutons, LEDs, buzzer, écran LCD, et envoi des votes sur le port série
<code>firmware/README.md</code>	Instructions de flash, librairies requises, points de débogage connus
<code>bridge/serial_to_api.py</code>	Script Python qui lit le port série et relaie chaque vote vers l'API backend (<code>POST /api/votes</code>)
<code>bridge/requirements.txt</code>	Dépendances Python : <code>pyserial</code> , <code>requests</code> , <code>python-dotenv</code>
<code>bridge/.env.example</code> / <code>.env</code>	Configuration : port série, débit, URL du backend, lieu de la borne (optionnel)
<code>bridge/README.md</code>	Installation, configuration, lancement du bridge

3.2 Documentation générale

- `docs/boitier-arduino.md` — spécification du boîtier physique (dimensions 180×160×90 mm, façade inclinée, LCD centré, boutons colorés). La modélisation 3D et l'impression restent à faire — hors de portée de cette session.
- `REQUIREMENTS.md` — décision « Borne IoT → simulée en logiciel » remplacée par « Borne IoT → prototype matériel Arduino Uno » ; nouvelle section « Matériel » avec la liste des composants et le tableau de brochage ; ligne « Budget » nuancée pour approuver la dépense matérielle ; retrait de « déploiement sur un vrai dispositif physique » de la liste hors périmètre.
- `README.md` — diagramme d'architecture mis à jour avec le bloc « Borne IoT (Arduino Uno) » et le bridge Python ; tableau de la stack technique complété ; section « État du projet » mise à jour.

4. Découverte : le montage réel diffère du plan initial

Point clé de la soirée. Le premier firmware écrit supposait un écran LCD avec module I2C (4 fils : GND/VCC/SDA/SCL). Le montage réel de l'utilisateur utilise un **LCD câblé en mode parallèle** (RS, E, D4-D7 branchés directement sur l'Arduino), avec des broches différentes de celles prévues initialement.

L'utilisateur a fourni son propre code Arduino déjà fonctionnel (testé physiquement, IDE Arduino). Le firmware du dépôt a été entièrement remplacé par ce code réel, en n'y ajoutant que le strict nécessaire : l'envoi du vote sur le port série (`Serial.println("VOTE:...")`), pour rester fidèle au comportement déjà validé par l'utilisateur (textes LCD, réglages du buzzer, etc. inchangés).

Brochage réel (mis à jour dans REQUIREMENTS.md)

Composant	Broche
LED verte (Heureux)	D7
LED jaune (Neutre)	D9
LED rouge (Triste)	D10
Bouton Heureux	D13
Bouton Neutre	D6
Bouton Triste	A4
Buzzer	D8
LCD RS	D12
LCD E	D11
LCD D4 / D5 / D6 / D7	D5 / D4 / D3 / D2

5. Mise en place de l'environnement de test

Étape	Action
Détection matériel	L'Arduino Uno a été détecté sur le port <code>COM7</code> (Gestionnaire de périphériques Windows)
Backend	Lancement du serveur FastAPI (<code>uvicorn main:app --host 0.0.0.0 --port 8000</code>), connecté à la base MySQL locale (celle déjà utilisée pour les votes web, contenant l'historique complet du projet)

Bridge	Création d'un environnement virtuel Python dédié, installation des dépendances, configuration de <code>.env</code> avec <code>SERIAL_PORT=COM7</code> et <code>BACKEND_URL=http://localhost:8000</code>
--------	---

6. Débogage de la connexion — chronologie des blocages

Trois obstacles successifs ont été rencontrés et résolus avant d'obtenir une connexion stable de bout en bout. Ils sont documentés ici en détail car ils se reproduiront probablement lors de futures reconnections.

Blocage n°1 — Mauvais fichier ouvert dans l'IDE Arduino

Premier test : aucun vote n'arrivait au bridge, alors que la LED et le LCD réagissaient normalement aux boutons. Diagnostic par capture d'écran de l'IDE Arduino : le fichier réellement ouvert et flashé s'appelait `sketch_aug4a.ino` — un fichier local du sketchbook par défaut de l'IDE, distinct du fichier `moodbar-iot/firmware/moodbar_kiosk.ino` du dépôt Git. Le code flashé était donc l'ancienne version, sans l'instruction d'envoi série.

Résolution : le code mis à jour (avec `Serial.println`) a été copié-collé directement dans `sketch_aug4a.ino` depuis l'IDE, puis re-téléversé. Vérifié via le Moniteur série : les lignes `VOTE:content`, `VOTE:neutre`, `VOTE:pas_content` apparaissaient bien à chaque appui.

Blocage n°2 — Deux instances du bridge tournant simultanément

Après correction du firmware, le bridge Python affichait en boucle `PermissionError(13, 'Accès refusé')` en tentant d'ouvrir `COM7`. Diagnostic : deux processus `serial_to_api.py` tournaient en parallèle (lancés lors d'essais successifs), se disputant l'accès exclusif au port série.

Résolution : arrêt de tous les processus Python du bridge, puis relance d'une seule instance propre.

Blocage n°3 — L'IDE Arduino retient le port même après fermeture de l'onglet

Le blocage persistait malgré une seule instance du bridge. Diagnostic : l'IDE Arduino 2.x fait tourner un service en arrière-plan (`arduino-cli`, visible comme processus séparé) qui garde la main sur le port série même après la fermeture du seul onglet « Moniteur série » — il faut fermer l'application entière.

Résolution : fermeture complète de l'IDE Arduino. Le bridge, qui retente la connexion automatiquement toutes les 3 secondes, s'est reconnecté avec succès (`Port série connecté.`).

7. Test de bout en bout réussi

Une fois les trois blocages résolus, l'appui sur le bouton « Heureux » de la borne physique a produit la chaîne complète attendue :

Bouton pressé

- LED verte allumée + buzzer + LCD "Humeur: HEUREUX :)"
- Arduino envoie "VOTE:content" sur le port série
- bridge Python lit la ligne, POST /api/votes {"humeur":"content","source":"iot"}
- backend FastAPI insère le vote en base MySQL
- confirmation "Vote transmis : content" dans le terminal du bridge

Vérification directe en base de données (requête SQL) :

```
SELECT id, humeur, horodatage, source FROM votes  
WHERE source = 'iot' ORDER BY horodatage DESC;
```

```
id=148  content  2026-08-04 21:53:36  iot  
id=147  content  2026-08-04 21:44:09  iot  
id=146  content  2026-08-04 21:44:07  iot
```

3 votes physiques confirmés en base, avec la source correctement identifiée comme `iot` — la borne réelle alimente désormais la même base de données que l'application web, exactement comme prévu par l'architecture du projet.

8. Consultation des données

Les données ont été consultées de deux façons complémentaires :

- **Ligne de commande MySQL** (`mysql.exe` , installé localement) — requêtes SQL directes, utile pour du diagnostic rapide et précis.
- **Dashboard Streamlit** (`moodbar-dashboard/app.py` , relancé sur `http://localhost:8501`) — vue agrégée avec métriques, donut et diagramme en bandes, incluant désormais automatiquement les votes `iot` mélangés aux votes `web` (144 votes web / 3 votes iot au moment de la vérification). Le dashboard ne distingue pas encore visuellement les deux sources — piste d'amélioration identifiée mais non traitée ce soir.

9. Checklist pour la prochaine reconnexion

Le firmware reste en mémoire sur l'Arduino même débranché — pas besoin de le reflasher tant que le code ne change pas. Pour reconnecter la borne un autre jour :

1. Brancher l'Arduino en USB.
2. **Ne pas ouvrir** le Moniteur série de l'IDE Arduino par réflexe — et si besoin de vérifier quelque chose avec, **fermer complètement l'application** ensuite (pas seulement l'onglet).
3. Vérifier le port COM dans le Gestionnaire de périphériques (peut changer selon le port USB utilisé) et mettre à jour `moodbar-iot/bridge/.env` si besoin.
4. Lancer le backend : `venv\Scripts\python.exe -m uvicorn main:app --host 0.0.0.0 --port 8000` (depuis `moodbar-backend/`).
5. Lancer le bridge : `venv\Scripts\python.exe serial_to_api.py` (depuis `moodbar-iot/bridge/`), attendre `Port série connecté.`
6. Tester un bouton, confirmer `Vote transmis : ...` dans le terminal du bridge.

10. Bilan et pistes restantes

Fait ce soir	État
Décision « borne IoT physique » actée dans la documentation	✅ Terminé
Firmware Arduino aligné sur le montage réel	✅ Terminé, validé sur matériel
Bridge série → API	✅ Terminé, validé de bout en bout
Vote physique → base de données	✅ Confirmé (3 votes tests)
Consultation dashboard	✅ Fonctionnel (sans filtre par source)
Modélisation 3D / impression du boîtier	⌚ À faire (hors de portée logicielle)
Compteur de votes affiché sur le LCD	⌚ Non demandé ce soir, évoqué comme piste
Filtre web/iot dans le dashboard	⌚ Proposé, pas encore développé
Commit / push de <code>moodbar-iot/</code> sur GitHub	⌚ En attente de confirmation de l'utilisateur

